

CCF 全国青少年信息学奥林匹克联赛

CCF NOIP 2025

时间：2026 年 13 月 32 日 08:30 ~ 13:00

题目名称	糖果店	清仓甩卖	树的价值	序列询问
题目类型	传统型	传统型	传统型	传统型
目录	candy	sale	tree	query
可执行文件名	candy	sale	tree	query
输入文件名	candy.in	sale.in	tree.in	query.in
输出文件名	candy.out	sale.out	tree.out	query.out
每个测试点时限	1.0 秒	1.0 秒	2.0 秒	2.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	20	25	25	20
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	candy.cpp	sale.cpp	tree.cpp	query.cpp
-----------	-----------	----------	----------	-----------

编译选项

对于 C++ 语言	-O2 -std=c++14 -static
-----------	------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考各省的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。
8. 全国统一评测时采用的机器配置为：Intel(R) Core(TM) i7-8700K CPU @3.70GHz，内存 32GB。上述时限以此配置为准。
9. 只提供 Linux 格式附加样例文件。
10. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

糖果店 (candy)

【题目描述】

小 X 的糖果店中有 n 颗糖果，其中第 i 颗糖果的价格为 a_i 元。小 R 带着 m 元来到糖果店，可以任意选择若干颗糖果购买。

为了让顾客能够一次品尝两种口味，小 X 准备了至多 k 个双糖礼盒。每个双糖礼盒必须装入两颗已经购买且没有装入其他双糖礼盒的糖果。装入同一个双糖礼盒的两颗糖果不再分别结算，这个双糖礼盒的价格等于其中较贵糖果的价格。没有装入双糖礼盒的糖果仍按原价结算。

小 R 希望购买的糖果尽可能多，同时支付的总费用不能超过 m 元。

你需要求出小 R 最多能够购买多少颗糖果。

【输入格式】

本题包含多组测试数据。

从文件 `candy.in` 中读入数据。

输入的第一行包含两个非负整数 c, T ，分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。

接下来依次输入每组测试数据。对于每组测试数据：

第一行包含三个非负整数 n, m, k ，分别表示糖果数量、小 R 拥有的钱数与最多能够使用的双糖礼盒数量。

第二行包含 n 个正整数 $a_1, a_2, \dots, a_n$ ，其中 a_i 表示第 i 颗糖果的价格。

【输出格式】

输出到文件 `candy.out` 中。

对于每组测试数据，输出一行一个非负整数，表示小 R 最多能够购买的糖果数量。

【样例 1 输入】

```
1 0 2
2 5 11 1
3 2 4 7 3 6
4 6 16 2
5 8 5 6 3 4 9
```

【样例 1 输出】

```
1 4
2 5
```

【说明/提示】**【样例 1 解释】**

对于第一组测试数据，小 R 可以购买价格分别为 2, 3, 4, 6 元的四颗糖果，并将价格为 4, 6 元的两颗糖果装入一个**双糖礼盒**。总费用为 $2 + 3 + 6 = 11$ 元。

可以证明，购买五颗糖果至少需要 16 元，因此答案为 4。

对于第二组测试数据，小 R 可以购买价格分别为 3, 4, 5, 6, 8 元的五颗糖果，将价格为 4, 5 元与价格为 6, 8 元的糖果分别装入两个**双糖礼盒**，并单独购买价格为 3 元的糖果。总费用为 $3 + 5 + 8 = 16$ 元。

可以证明，购买六颗糖果至少需要 20 元，因此答案为 5。

【样例 2】

见选手目录下的 *candy/candy2.in* 和 *candy/candy2.ans*。
该附加样例满足测试点 1 ~ 3 的数据范围。

【样例 3】

见选手目录下的 *candy/candy3.in* 和 *candy/candy3.ans*。
该附加样例满足测试点 4 ~ 6 的数据范围。

【样例 4】

见选手目录下的 *candy/candy4.in* 和 *candy/candy4.ans*。
该附加样例满足测试点 10 ~ 15 的数据范围。

【样例 5】

见选手目录下的 *candy/candy5.in* 和 *candy/candy5.ans*。
该附加样例满足测试点 16 ~ 20 的数据范围。

【数据范围】

设 N 为单个测试点内所有测试数据的 n 之和。

测试点编号	$n \leq$	特殊性质
1 ~ 3	10	无
4 ~ 6	2×10^5	$k = 0$
7 ~ 9		$k \leq 1$
10 ~ 15	2×10^3	无
16 ~ 20	2×10^5	

对于所有测试数据，保证：

- $0 \leq c \leq 20$, $1 \leq T \leq 2 \times 10^4$ 。
- $1 \leq n \leq 2 \times 10^5$, $N \leq 5 \times 10^5$ 。
- $0 \leq k \leq \lfloor n/2 \rfloor$ 。
- $0 \leq m \leq 10^{18}$ 。
- 对于所有 $1 \leq i \leq n$ ，均有 $1 \leq a_i \leq 10^9$ 。

清仓甩卖 (sale)

【题目背景】

Celia 逛超市时，经常喜欢挑选性价比更高的商品，俗称为“物美价廉”。

【题目描述】

小 X 的促销计划十分成功，现在糖果店中只剩下了 n 颗糖果，其中第 i 颗糖果的原价为 w_i 元。为了尽快清空余货，小 X 为每颗糖果标出了清仓价格。第 i 颗糖果的清仓价格为 v_i 元，其中 $v_i \in \{1, 2\}$ 。

小 R 带着 m 元来到糖果店，可以任意选择若干颗糖果购买。所选糖果的清仓价格之和称为这次购买的总价格，原价之和称为这次购买的总价值。小 R 要求总价格不超过 m ，并希望总价值尽可能大。

你需要帮助小 R 求出能够得到的最大总价值。

【输入格式】

本题包含多组测试数据。

从文件 `sale.in` 中读入数据。

输入的第一行包含两个非负整数 c, T ，分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。

接下来依次输入每组测试数据。对于每组测试数据：

第一行包含两个正整数 n, m ，分别表示糖果数量与小 R 拥有的钱数。

第二行包含 n 个正整数 $w_1, w_2, \dots, w_n$ ，其中 w_i 表示第 i 颗糖果的原价。

第三行包含 n 个正整数 $v_1, v_2, \dots, v_n$ ，其中 v_i 表示第 i 颗糖果的清仓价格。

【输出格式】

输出到文件 `sale.out` 中。

对于每组测试数据，输出一行一个非负整数，表示小 R 能够得到的最大总价值。

【样例 1 输入】

```
1 0 1
2 5 5
3 8 7 10 5 6
4 1 2 2 1 2
```

【样例 1 输出】

125

【说明/提示】

【样例 1 解释】

小 R 可以购买第 1, 2, 3 颗糖果。这次购买的总价格为 $1 + 2 + 2 = 5$ ，总价值为 $8 + 7 + 10 = 25$ 。

可以证明，不存在总价值更大的合法购买方案。

【样例 2】

见选手目录下的 *sale/sale2.in* 和 *sale/sale2.ans*。

该附加样例满足测试点 1 ~ 4 的数据范围。

【样例 3】

见选手目录下的 *sale/sale3.in* 和 *sale/sale3.ans*。

该附加样例满足测试点 9 ~ 12 的数据范围。

【样例 4】

见选手目录下的 *sale/sale4.in* 和 *sale/sale4.ans*。

该附加样例满足测试点 13 ~ 19 的数据范围。

【样例 5】

见选手目录下的 *sale/sale5.in* 和 *sale/sale5.ans*。

该附加样例满足测试点 20 ~ 25 的数据范围。

【数据范围】

设 N 为单个测试点内所有测试数据的 n 之和。

测试点编号	$n \leq$	特殊性质
1 ~ 4	20	无
5 ~ 8	2×10^5	对于所有 i ，均有 $v_i = 1$
9 ~ 12		对于所有 i ，均有 $v_i = 2$
13 ~ 19	2×10^3	无
20 ~ 25	2×10^5	

对于所有测试数据，保证：

- $0 \leq c \leq 25, 1 \leq T \leq 5 \times 10^4$ 。

- $1 \leq n \leq 2 \times 10^5$, $N \leq 5 \times 10^5$ 。
- $1 \leq m \leq 2n$ 。
- 对于所有 $1 \leq i \leq n$, 均有 $1 \leq w_i \leq 10^9$, $v_i \in \{1, 2\}$ 。

树的价值 (tree)

【题目描述】

给定一棵 n 个结点的有根树，其中结点 1 为根，结点 i ($2 \leq i \leq n$) 的父亲结点为结点 p_i 。

对于 $1 \leq i \leq n$ ，定义结点 i 的深度 d_i 为结点 1 到结点 i 的简单路径的边数。也就是说， $d_1 = 0$ ， $d_i = d_{p_i} + 1$ ($2 \leq i \leq n$)。定义有根树的高度 h 为所有结点深度的最大值，即 $h = \max_{i=1}^n d_i$ 。

给定高度的上界 m 。在本题中，给定的有根树的高度不超过 m 。

每个结点 u 有一个整数权值 a_u 。另外，给定一个长度为 n 的整数序列 $b_1, b_2, \dots, b_n$ ，其中 b_j 表示长度为 j 的链对应的**价值系数**。权值 a_u 与**价值系数** b_j 均可能为负数。

若一个结点没有儿子，则称这个结点为叶子。设树中共有 k 个叶子。小 X 会选择一个由所有叶子组成的排列 $q_1, q_2, \dots, q_k$ ，并按照这个顺序依次探索这些叶子。这个排列称为一个**探索序列**。

当小 X 探索叶子 q_i 时，会观察从根到 q_i 的简单路径，并取出这条路径上此前从未被探索过的所有结点，记这些结点组成的集合为 S_i 。此前已经探索过的所有根到叶路径构成一棵包含根的连通子树，因此 S_i 中的结点一定构成一条连续的、从上向下的链，称为第 i 次探索的**新探索链**。

令 t_i 表示 S_i 中深度最小的结点， $l_i = |S_i|$ 表示这条**新探索链**的长度。定义第 i 次探索产生的**探索价值**为 $a_{t_i} b_{l_i}$ ，并定义这个**探索序列**的总价值为

$$\sum_{i=1}^k a_{t_i} b_{l_i}.$$

定义这棵树的**价值**为所有**探索序列**的总价值的最大值。

你需要求出给定有根树的**价值**。

【输入格式】

本题包含多组测试数据。

从文件 `tree.in` 中读入数据。

输入的第一行包含一个非负整数 c 与一个正整数 T ，分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。

接下来依次输入每组测试数据。对于每组测试数据：

第一行包含两个正整数 n, m ，分别表示结点数量与高度的上界。

第二行包含 $n - 1$ 个正整数 $p_2, p_3, \dots, p_n$ ，分别表示每个结点的父亲结点。

第三行包含 n 个整数 $a_1, a_2, \dots, a_n$ ，分别表示每个结点的权值。

第四行包含 n 个整数 $b_1, b_2, \dots, b_n$ ，分别表示不同链长对应的**价值系数**。

【输出格式】

输出到文件 *tree.out* 中。

对于每组测试数据，输出一行一个整数，表示给定有根树的**价值**。

【样例 1 输入】

```
1 0 2
2 5 2
3 1 1 2 2
4 3 -2 4 5 1
5 2 -1 3 0 0
6 4 3
7 1 2 3
8 -3 2 -1 5
9 7 0 2 -4
```

【样例 1 输出】

```
1 27
2 12
```

【说明/提示】**【样例 1 解释】**

对于第一组测试数据，树中的叶子为结点 3, 4, 5。

小 X 可以选择探索序列 5, 3, 4。第一次探索叶子 5 时， $S_1 = \{1, 2, 5\}$ ，产生的探索价值为 $a_1 b_3 = 3 \times 3 = 9$ 。第二次探索叶子 3 时， $S_2 = \{3\}$ ，产生的探索价值为 $a_3 b_1 = 4 \times 2 = 8$ 。第三次探索叶子 4 时， $S_3 = \{4\}$ ，产生的探索价值为 $a_4 b_1 = 5 \times 2 = 10$ 。

这个探索序列的总价值为 $9 + 8 + 10 = 27$ 。可以证明，不存在总价值更大的探索序列，因此这棵树的价值为 27。

对于第二组测试数据，结点 4 是唯一的叶子。唯一的探索序列为 4，此时 $S_1 = \{1, 2, 3, 4\}$ ，产生的探索价值为 $a_1 b_4 = (-3) \times (-4) = 12$ ，因此这棵树的价值为 12。

【样例 2】

见选手目录下的 *tree/tree2.in* 和 *tree/tree2.ans*。

该附加样例满足测试点 3, 4 的数据范围。

【样例 3】

见选手目录下的 *tree/tree3.in* 和 *tree/tree3.ans*。

该附加样例满足测试点 13, 14 的数据范围。

【样例 4】

见选手目录下的 *tree/tree4.in* 和 *tree/tree4.ans*。

该附加样例满足测试点 18, 19 的数据范围。

【样例 5】

见选手目录下的 *tree/tree5.in* 和 *tree/tree5.ans*。

该附加样例满足测试点 20 ~ 25 的数据范围。

【数据范围】

测试点编号	$n \leq$	$m \leq$
1, 2	7	$n - 1$
3, 4	13	
5, 6	18	
7, 8	40	
9, 10	120	
11, 12	360	
13, 14	4×10^3	2
15 ~ 17		10
18, 19		50
20 ~ 25		800

对于所有测试数据，保证：

- $0 \leq c \leq 25$, $1 \leq T \leq 5$ 。
- $2 \leq n \leq 8 \times 10^3$, $1 \leq m \leq \min(n - 1, 800)$ 。
- 对于所有 $2 \leq i \leq n$ ，均有 $1 \leq p_i \leq i - 1$ 。
- 给定的有根树的高度不超过 m 。
- 对于所有 $1 \leq i \leq n$ ，均有 $-10^6 \leq a_i, b_i \leq 10^6$ 。

序列询问 (query)

【题目描述】

给定一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 其中对于所有 $1 \leq i \leq n$, 均有 $a_i \in \{0, 1\}$ 。

对于任意一个只包含 0, 1 的序列 $S = (s_1, s_2, \dots, s_k)$, 从左到右依次扫描其中的元素。初始时栈为空。若栈非空且当前元素与栈顶元素相同, 则弹出栈顶, 否则将当前元素压入栈中。扫描结束后, 将栈中元素从栈底到栈顶依次排列, 得到的消除结果记为 $\text{red}(S)$ 。

例如, 当 $S = (0, 1, 1, 0)$ 时, 栈中的元素依次变化为 $\emptyset \rightarrow 0 \rightarrow 01 \rightarrow 0 \rightarrow \emptyset$, 因此 $\text{red}(S) = \emptyset$ 。

有 q 次询问, 其中第 j 次询问给出两个整数 l_j, r_j 。对于每个满足 $l_j \leq m < r_j$ 的整数 m , 定义左侧序列 $L_{j,m} = (a_{l_j}, a_{l_j+1}, \dots, a_m)$, 右侧序列 $R_{j,m} = (a_{m+1}, a_{m+2}, \dots, a_{r_j})$ 。

对于两个序列 A, B , 记 $A \circ B$ 表示将 B 接在 A 后得到的序列。先分别求出 $\text{red}(L_{j,m})$ 与 $\text{red}(R_{j,m})$, 再将二者拼接, 得到 $C_{j,m} = \text{red}(L_{j,m}) \circ \text{red}(R_{j,m})$ 。

此后继续消除 $C_{j,m}$ 。这一阶段新消除的元素对数称为分割点 m 的拼接消除次数, 记为

$$f_j(m) = \frac{|C_{j,m}| - |\text{red}(C_{j,m})|}{2}.$$

特别地, 分别计算 $\text{red}(L_{j,m})$ 与 $\text{red}(R_{j,m})$ 时发生的消除不计入 $f_j(m)$ 。

对于每次询问, 你需要求出所有分割点的拼接消除次数之和, 即

$$\sum_{m=l_j}^{r_j-1} f_j(m).$$

【输入格式】

从文件 `query.in` 中读入数据。

本题有多组测试数据。

输入的第一行包含两个整数 c, T , 分别表示测试点编号和测试数据的组数。 $c = 0$ 表示该测试点为样例。

对于每组测试数据:

- 第一行包含两个正整数 n, q , 分别表示序列长度与询问次数。
- 第二行包含 n 个整数 $a_1, a_2, \dots, a_n$, 描述给定的序列。
- 接下来 q 行, 每行包含两个整数 l, r , 表示一次询问的区间。

【输出格式】

输出到文件 `query.out` 中。

对于每次询问，输出一行一个非负整数，表示所有分割点的**拼接消除次数**之和。

【样例 1 输入】

```
1 0 1
2 5 3
3 0 1 1 0 1
4 1 4
5 1 5
6 2 5
```

【样例 1 输出】

```
1 4
2 4
3 1
```

【说明/提示】**【样例 1 解释】**

考虑第一次询问 $[1, 4]$ 。

当 $m = 1$ 时， $\text{red}(L_{1,1}) = (0)$ ， $\text{red}(R_{1,1}) = (0)$ ，因此 $f_1(1) = 1$ 。

当 $m = 2$ 时， $\text{red}(L_{1,2}) = (0, 1)$ ， $\text{red}(R_{1,2}) = (1, 0)$ ，因此 $f_1(2) = 2$ 。

当 $m = 3$ 时， $\text{red}(L_{1,3}) = (0)$ ， $\text{red}(R_{1,3}) = (0)$ ，因此 $f_1(3) = 1$ 。

因此，这次询问的答案为 $1 + 2 + 1 = 4$ 。

【样例 2】

见选手目录下的 *query/query2.in* 和 *query/query2.ans*。

该附加样例满足测试点 1 ~ 3 的数据范围。

【样例 3】

见选手目录下的 *query/query3.in* 和 *query/query3.ans*。

该附加样例满足测试点 7 ~ 9 的数据范围。

【样例 4】

见选手目录下的 *query/query4.in* 和 *query/query4.ans*。

该附加样例满足测试点 10 ~ 15 的数据范围。

【样例 5】

见选手目录下的 `query/query5.in` 和 `query/query5.ans`。

该附加样例满足测试点 16 ~ 20 的数据范围。

【数据范围】

测试点编号	$n \leq$	$q \leq$
1 ~ 3	30	30
4 ~ 6	2×10^3	2×10^3
7 ~ 9	10^5	30
10 ~ 15	5×10^3	5×10^3
16 ~ 20	10^5	10^5

对于所有测试数据，保证：

- $0 \leq c \leq 20$ 。
- $1 \leq T \leq 10$ 。
- 对于每组测试数据，均有 $2 \leq n \leq 10^5$ ， $1 \leq q \leq 10^5$ 。
- 对于单个测试点内的所有测试数据，保证 $\sum n \leq 10^5$ ， $\sum q \leq 10^5$ 。
- 对于所有 $1 \leq i \leq n$ ，均有 $a_i \in \{0, 1\}$ 。
- 对于每次询问，均有 $1 \leq l < r \leq n$ 。